

DAY 4 BACKEND TASKS

Vagdevi Malineni

malinenv24@gmail.com

Task 1: User Registration API

1. Add User Schema



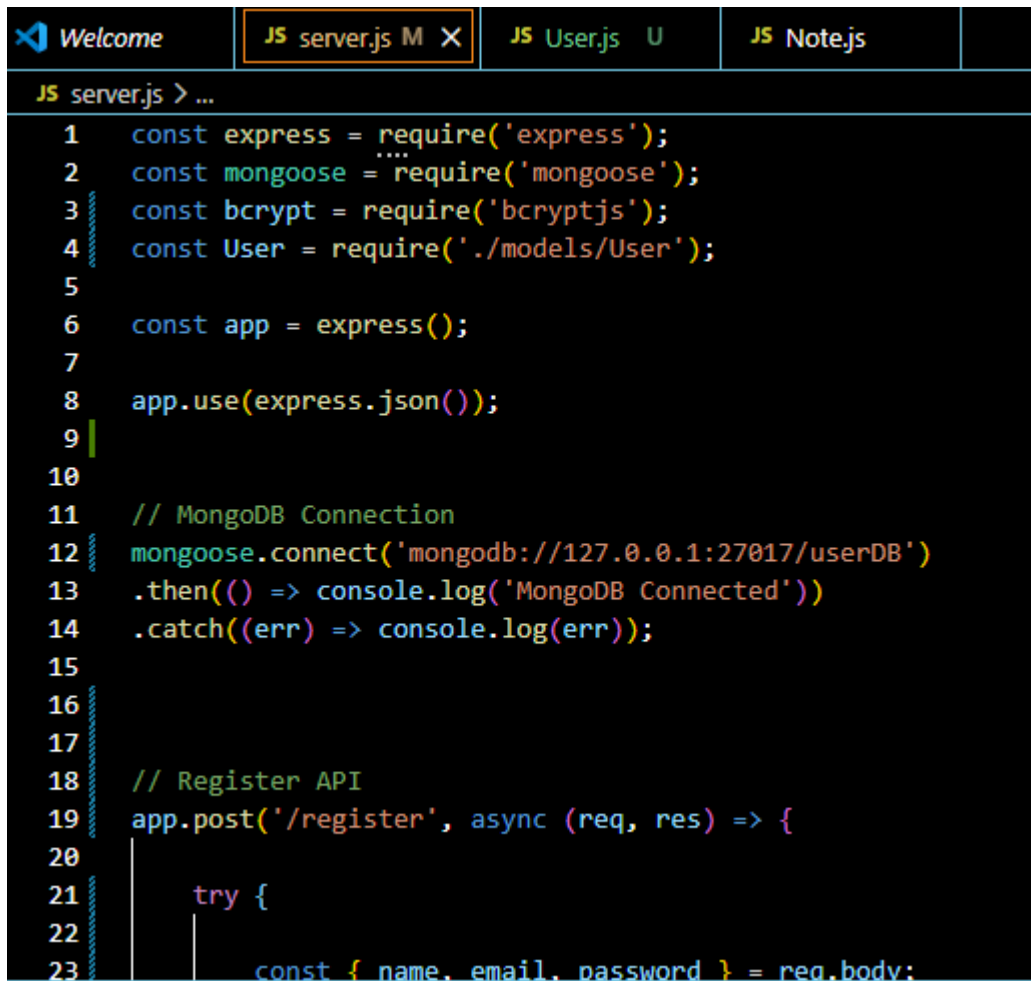
The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree with the following structure:

- BACKEND (expanded)
 - models
 - Note.js
 - User.js (selected, marked with a 'U')
 - node_modules
 - package-lock.json (M)
 - package.json (M)
 - server.js (M)

The main editor area shows the content of 'models > JS User.js > ...' with the following code:

```
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema({
4    name: String,
5    email: String,
6    password: String
7  });
8
9  module.exports = mongoose.model('User', userSchema);
```

2. Create server.js



```
JS server.js M X JS User.js U JS Note.js
JS server.js > ...
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const bcrypt = require('bcryptjs');
4  const User = require('./models/User');
5
6  const app = express();
7
8  app.use(express.json());
9
10
11  // MongoDB Connection
12  mongoose.connect('mongodb://127.0.0.1:27017/userDB')
13  .then(() => console.log('MongoDB Connected'))
14  .catch((err) => console.log(err));
15
16
17
18  // Register API
19  app.post('/register', async (req, res) => {
20
21      try {
22
23          const { name, email, password } = req.body;
```

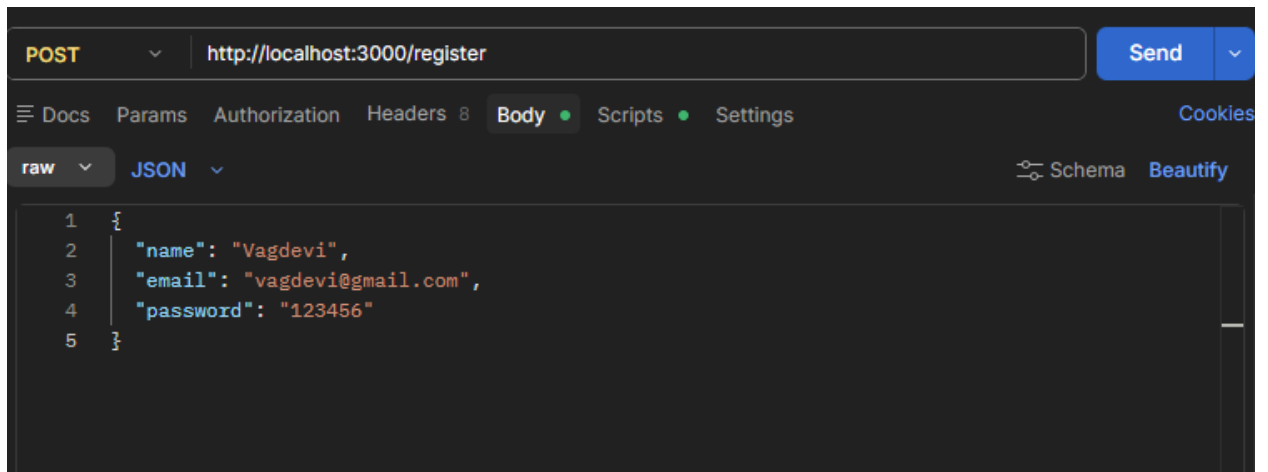
3. Run Server

```
PS C:\Users\V KRISHNA\backend-project> npm install express mongoose bcryptjs
added 1 package, and audited 84 packages in 2s

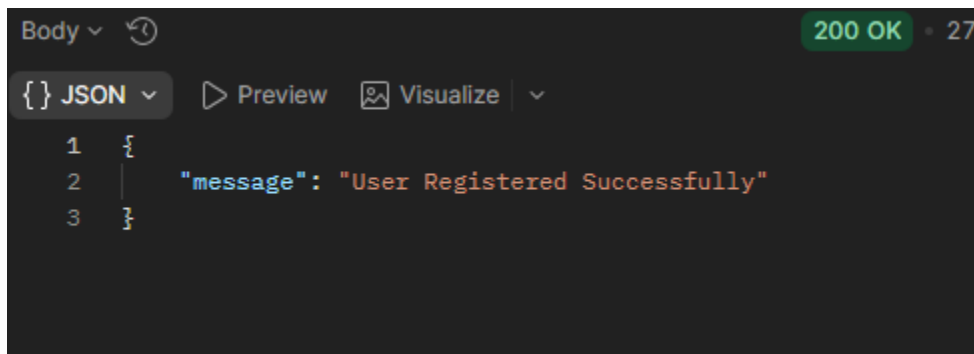
23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\V KRISHNA\backend-project> node server.js
Server running on port 3000
MongoDB Connected
█
```

4. Test API in Postman



OUTPUT:



5. Verify Data in MongoDB

Output: will see password in hash format

```

PS C:\Users\V KRISHNA\backend-project> mongosh
Current Mongosh Log ID: 6a055e7c48ec631702abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      7.0.32
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
  The server generated these startup warnings when booting
  2026-05-13T11:18:13.227+05:30: Access control is not enabled for the database. Read and write access to data and c
onfiguration is unrestricted
-----

test> show dbs
admin      40.00 KiB
backendDB  40.00 KiB
config     72.00 KiB
local      72.00 KiB
mydb       40.00 KiB
userDB     40.00 KiB
test> use userDB
switched to db userDB
userDB> show collections
users
userDB> db.users.find()
[
  {
    _id: ObjectId('6a055e033b2cfd12c797bad'),
    name: 'Vagdevi',
    email: 'vagdevi@gmail.com',
    password: '$2b$10$VXE7z7FIFjoB10IwYD6pfeDVBpPyekLrGEKbR3cUvfLuMxzXKV48i',
    __v: 0
  }
]

```

Task 2 — User Login API

This API will:

- check user exists
- compare password with hashed password
- return proper response

1. Add Login API

```
// Login API
app.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

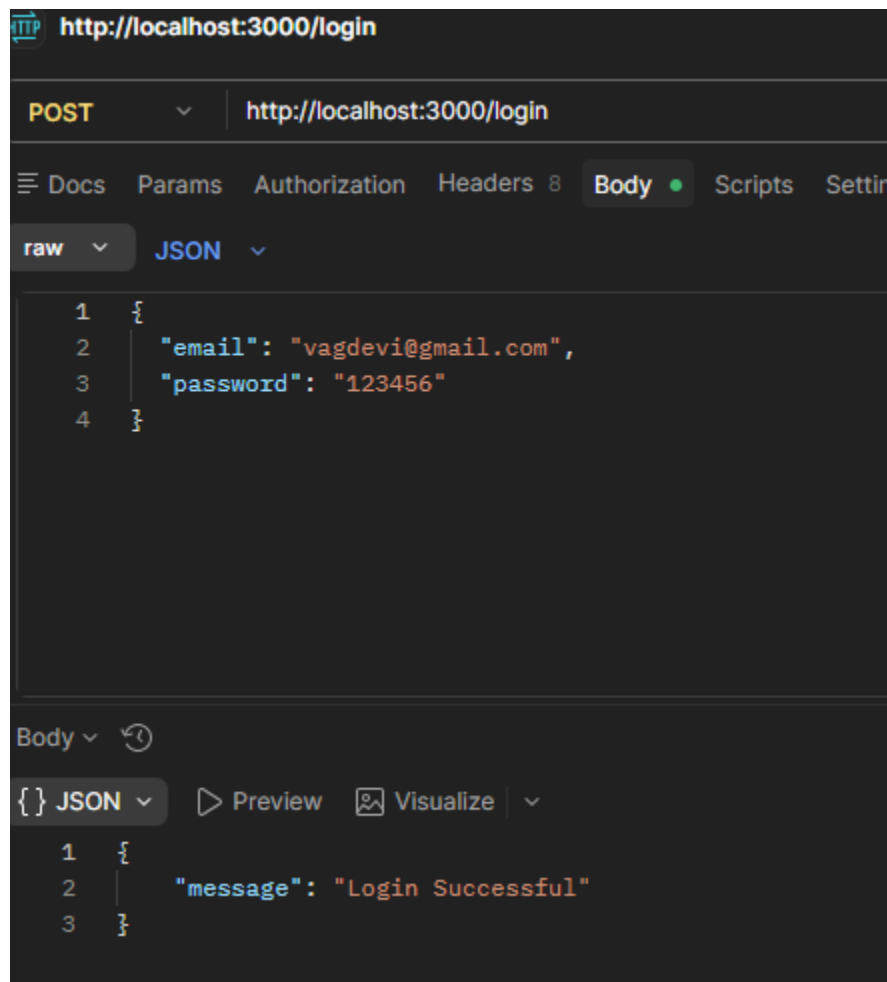
    const user = await User.findOne({ email });

    if (!user) {
      return res.json({
        message: 'User Not Found'
      });
    }

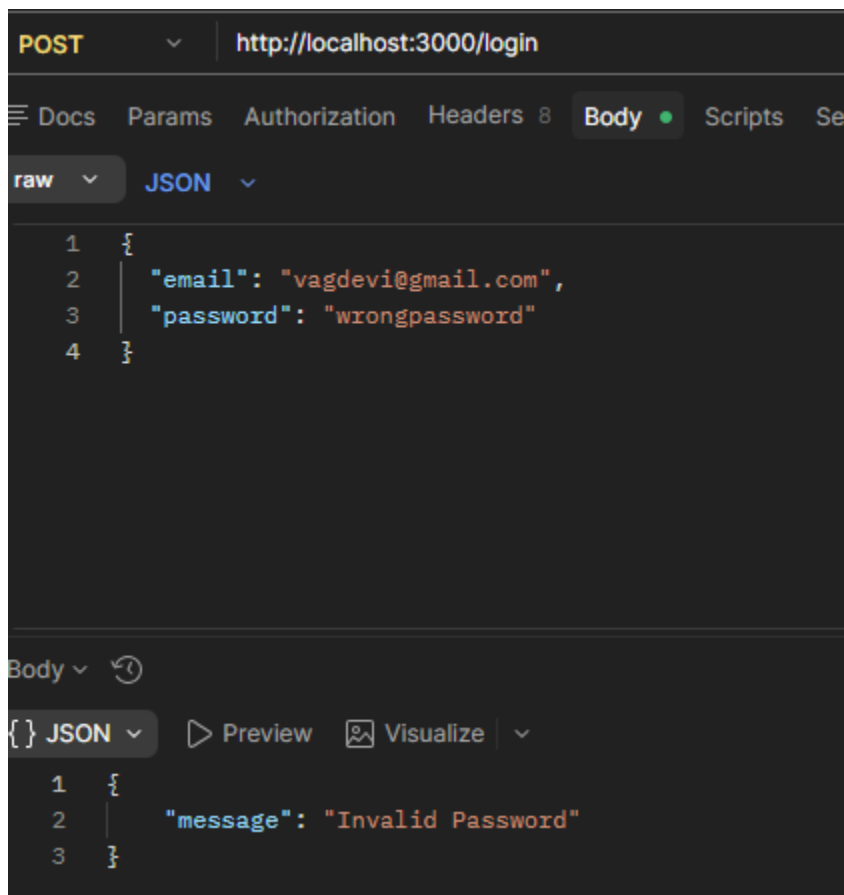
    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.json({
        message: 'Invalid Password'
      });
    }
  }
});
```

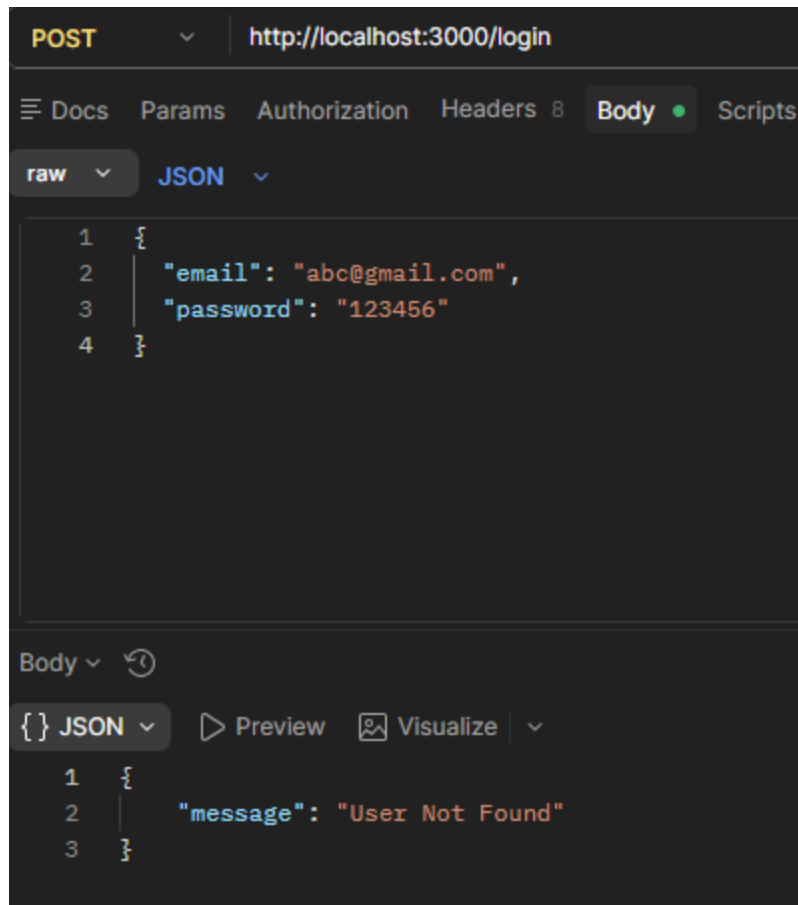
2. Test Login API in Postman



3. Test Invalid Password



4. Test User Not Found



authentication system is now working successfully

Task 3 — User Management APIs

will create: GET /users ,DELETE /user/:id

These APIs will:

fetch all users

delete user by ID

1. Open server.js

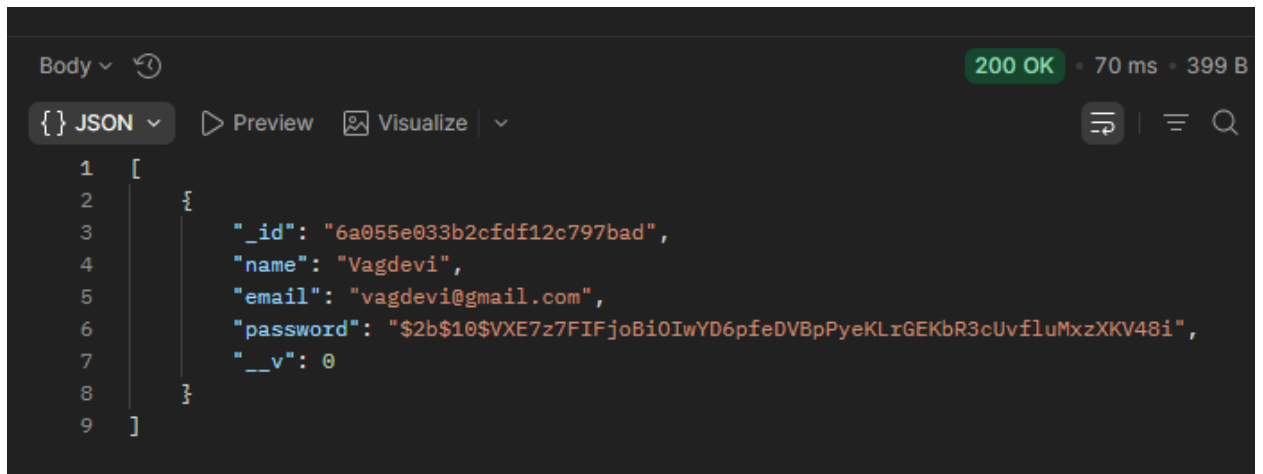
```
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const bcrypt = require('bcryptjs');
4  const User = require('./models/User');
5
6  const app = express();
7
8  app.use(express.json());
9
10
11  // MongoDB Connection
12  mongoose.connect('mongodb://127.0.0.1:27017/userDB')
13  .then(() => console.log('MongoDB Connected'))
14  .catch((err) => console.log(err));
15
16
17
18  // Test Route
19  app.get('/test', (req, res) => {
20    res.send('Test Working');
```

```
    app.post('/login', async (req, res) => {
      // Compare Password
      const isMatch = await bcrypt.compare(password, user.password);

      if (!isMatch) {
        return res.json({
          message: 'Invalid Password'
        });
      }

      res.json({
        message: 'Login Successful'
      });
    } catch (error) {
      res.status(500).json({
        message: 'Server Error'
      });
    }
  });
```

2. Test GET Users API in Postman



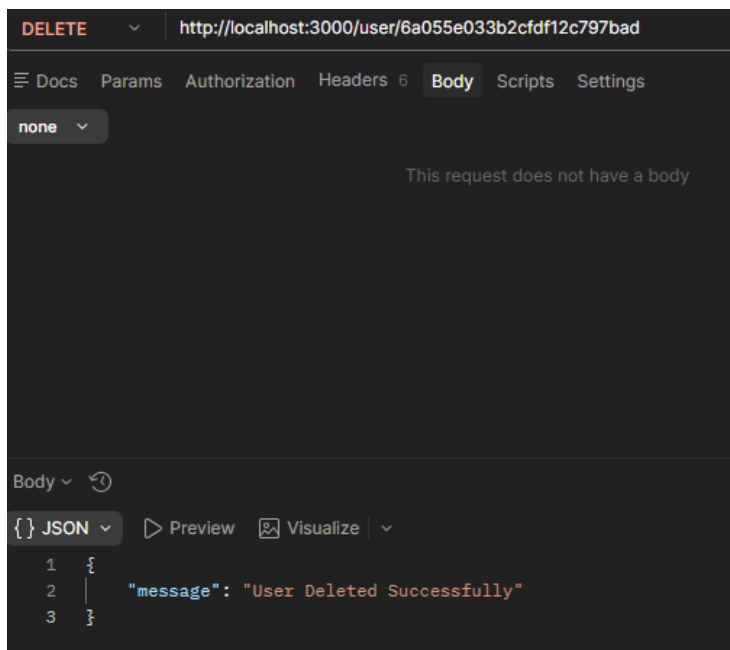
Body ▾ ↺

200 OK • 70 ms • 399 B

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1  [
2    {
3      "_id": "6a055e033b2cfd12c797bad",
4      "name": "Vagdevi",
5      "email": "vagdevi@gmail.com",
6      "password": "$2b$10$VXE7z7FIFjoBiOIwYD6pfeDVBpPyeKLrGEKbR3cUvfluMxzXKV48i",
7      "__v": 0
8    }
9  ]
```

3. Test DELETE API



DELETE ▾ http://localhost:3000/user/6a055e033b2cfd12c797bad

≡ Docs Params Authorization Headers 6 Body Scripts Settings

none ▾

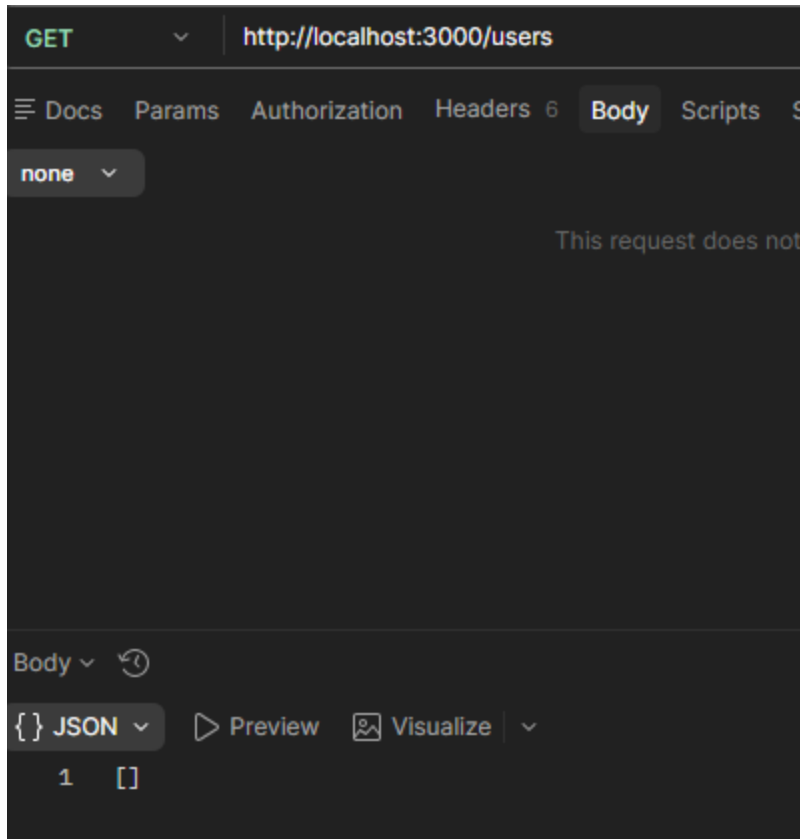
This request does not have a body

Body ▾ ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1  {
2    |   "message": "User Deleted Successfully"
3  }
```

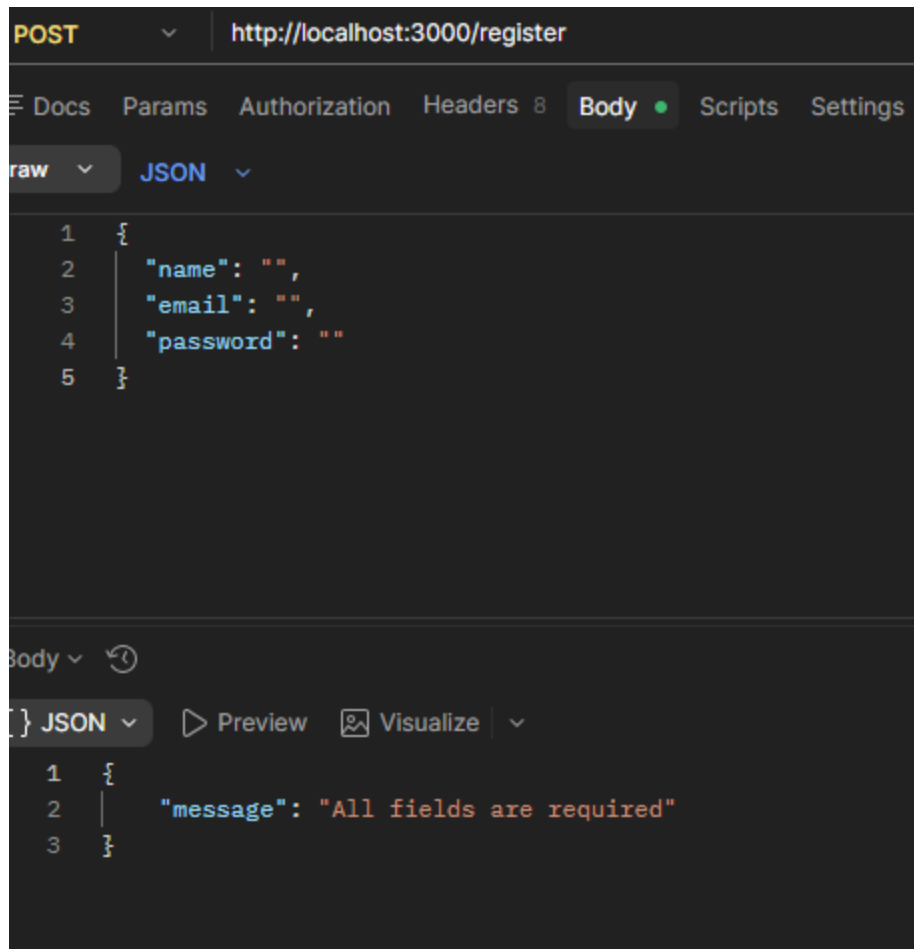
4. Verify Deletion



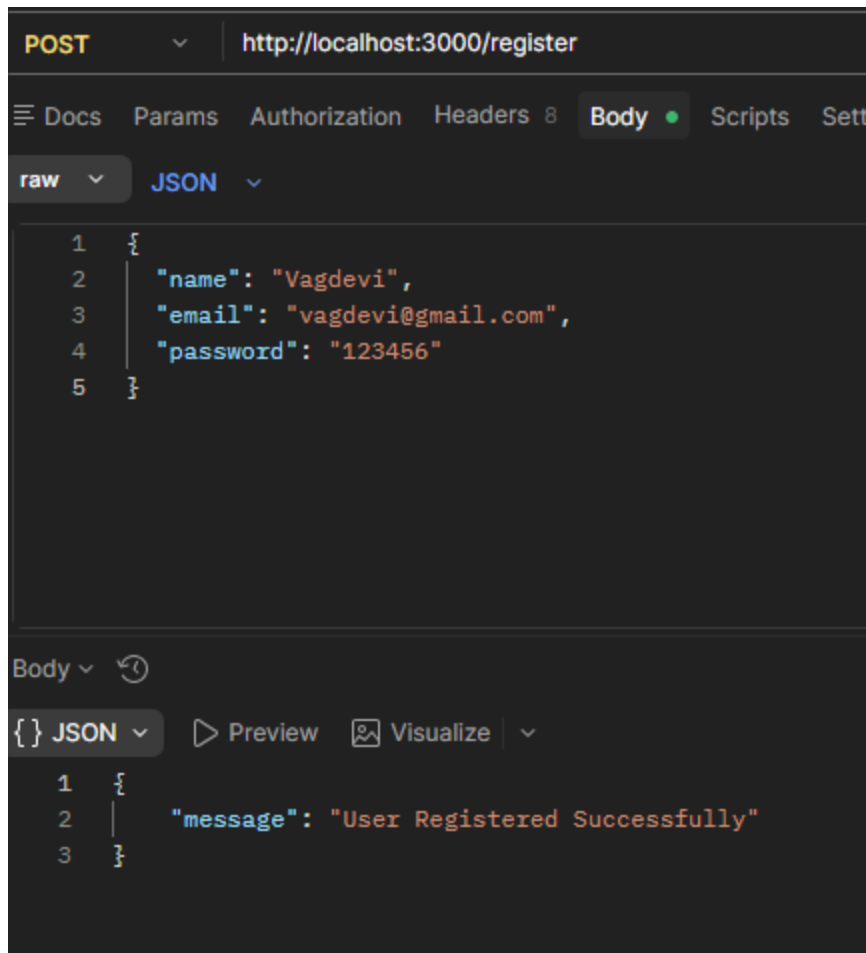
Task 4- Validation & Security

1. Test Cases in Postman

Empty Fields



2. Successful Registration



3. Duplicate Email

POST http://localhost:3000/register

Docs Params Authorization Headers 8 Body Scripts

raw JSON

```
1 {
2   "name": "Vagdevi",
3   "email": "vagdevi@gmail.com",
4   "password": "123456"
5 }
```

Body

{ } JSON Preview Visualize

```
1 {
2   "message": "Email already registered"
3 }
```

4. Password Less Than 6 Characters

raw JSON

```
1 {
2   "name": "Rahul",
3   "email": "rahul@gmail.com",
4   "password": "123"
5 }
```

Body

{ } JSON Preview Visualize

```
1 {
2   "message": "Password must contain at least 6 characters"
3 }
```

5. Login Success

```
1  {
2    "email": "vagdevi@gmail.com",
3    "password": "123456"
4  }
```

Body ▾ ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1  {
2    "message": "Login Successful"
3  }
```

6. Invalid Password

Body ▾ ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1  {
2    "message": "Invalid Password"
3  }
```

7. Get All Users

Body  200 OK • 25 ms • 399 B

{ } JSON   Preview  Visualize    

```
1  [  
2    {  
3      "_id": "6a05755976ec06413f47442c",  
4      "name": "Vagdevi",  
5      "email": "vagdevi@gmail.com",  
6      "password": "$2b$10$mQvi.uIyF6cueq8r.HV0Juh3cGmNu0hZnaq0fkjRQTtH1Vo2kAJ7C",  
7      "__v": 0  
8    }  
9  ]
```